

Nombre: *Jair Patiño*



Carga de datos.

In [10]:
```
X_train = np.array([[ 1., -4., 5.],
                    [ 5., 0., 2.],
                    [ 0., 6., -3.]])
print(X_train)
```

```
[[ 1. -4.  5.]
 [ 5.  0.  2.]
 [ 0.  6. -3.]]
```

Limpieza de datos: normalización.

In [11]:
```
normalizer = preprocessing.MinMaxScaler()
X_train_norm = normalizer.fit_transform(X_train)
```

Salida de resultados.

In [12]:
```
print(X_train_norm)
```

```
[[0.2   0.    1.   ]
 [1.    0.4   0.625]
 [0.    1.    0.   ]]
```

Ejemplo para ver que los resultados son iguales si aplicamos los métodos --> "fit" y "transform" por separado

In [13]:
```
X_train_2 = np.copy(X_train)
```

Crea un objeto "fitted_normalizer" llamando al método "fit"

In [14]:
```
f = normalizer.fit(X_train)
```

Aplica el objeto sobre los datos "X_train_2" para obtener "X_train_norm_2"

In [15]:
```
X_train_norm_2 = f.transform(X_train)
```

Comprueba si "X_train_norm" y "X_train_norm_2" son iguales

In [16]:
```
print(np.array_equiv(X_train_norm, X_train_norm_2))
```

```
True
```

Normalizar un nuevo conjunto de datos de test (SOLO HAY QUE APLICAR EL MÉTODO --> "transform", NO EL "fit")

In [17]:
```
X_test = np.array([[ 1., -3., 2.],
                   [ 3., 0., 0.],
                   [ 0., 4., -1.]])
```

Normalizar los datos de "X_test"

In [22]:
```
X_test_norm = normalizer.transform(X_test)
```

Salida de resultados

In [24]:
```
print(X_test_norm)
```

```
[[0.2   0.1   0.625]
 [0.6   0.4   0.375]
 [0.    0.8   0.25 ]]
```

GITHUB

- https://github.com/Failedbanjo/machine_learning_2